Протокол веб-разработки

Единый стандарт проектирования, разработки и сопровождения всех наших веб-проектов

Версия 1.0 · 2 ноября 2026 · Статус: черновик на утверждение · Пересмотр: по мере необходимости

> **Как пользоваться документом.** Это наш рабочий инженерный стандарт: по нему мы ведём каждый проект — от первой встречи с заказчиком до передачи и поддержки. Разделы 1–6 — фундамент и требования, раздел 11 — короткий чек-лист перед запуском, приложение — типовые стеки под задачу. Документ живой: правим и дополняем по мере роста практики.

Содержание

1. Базовые принципы
2. Жизненный цикл проекта
3. Архитектура и окружения
4. Технологический стек (2026)
5. Безопасность по максимуму
6. Персональные данные и приватность
7. Процесс разработки
8. Деплой и инфраструктура
9. Приёмка и сдача проекта
10. Поддержка, мониторинг, инциденты
11. Чек-лист перед запуском
12. Приложение: типовые стеки по типам проектов

1. Базовые принципы

Пять принципов, которым подчинено всё остальное. Если решение спорное — выбираем то, что сильнее соответствует этим принципам.

1. **Безопасность по умолчанию (security by default).** Защищённый вариант — вариант по умолчанию, а не опция «на потом». Любая новая функция оценивается на риск до того, как попадёт в прод.
2. **Простота и поддерживаемость.** Чем меньше движущихся частей, тем меньше ломается. Не тащим зависимость и не усложняем архитектуру, если задачу решает более простое средство.
3. **Воспроизводимость.** Любое окружение и любой деплой можно поднять заново по документации и из репозитория — без «магии в голове» и ручных недокументированных шагов.
4. **Всё задокументировано и забэкаплено.** Нет записи — считается, что этого нет. Перед любым рискованным изменением — бэкап. Бэкап без проверенного восстановления не считается бэкапом.
5. **Отказоустойчивость и независимость.** Критичные узлы не завязываем на единственного внешнего поставщика там, где это легко развести (яркий пример — DNS, см. раздел 4).

2. Жизненный цикл проекта

Каждый проект проходит одни и те же этапы. Нельзя перескакивать через архитектуру и безопасность ради скорости.

Этап	Что делаем	Результат этапа
1. Дискавери	Цели заказчика, аудитория, бюджет, сроки, ограничения, аналоги	Короткое резюме проекта + решение go/no-go
2. ТЗ и объём	Фиксируем функции, экраны, интеграции, что входит и что НЕ входит	Согласованное ТЗ
3. Архитектура	Выбор стека, схема данных, окружения, модель безопасности, план бэкапов	Архитектурная записка
4. Разработка	Итерации в git, бэкапы перед правками, самопроверка по чек-листам	Рабочая сборка на stage
5. Ревью и тест	Код-ревью, тест безопасности, проверка на реальных данных-образцах	Отчёт + закрытые замечания
6. Деплой	Выкатка на прод, проверка, прогрев, откат наготове	Живой проект
7. Сдача	Передача доступов, документации, обучение заказчика	Подписанная приёмка
8. Поддержка	Мониторинг, обновления, реакция на инциденты	Стабильная работа

3. Архитектура и окружения

3.1 Разделение окружений

- **dev** — локальная/рабочая разработка. **stage** — копия прода для проверки перед выкаткой. **prod** — боевой.
- Прод и stage максимально идентичны (версии, конфиг), различаются только данными и секретами.
- На проде никто не правит код «вживую». Любая правка проходит через git и выкатку.

3.2 Конфигурация и секреты

- Вся конфигурация — через переменные окружения (`.env`) или защищённый конфиг **вне веб-корня**. Никаких секретов в коде и в репозитории.
- Разные секреты для dev/stage/prod. Ротация ключей при смене подрядчика или утечке.
- Хранилище секретов (у нас — Vaultwarden) как единый источник правды для доступов.

3.3 Структура

- Явное разделение: фронтенд, API/бэкенд, данные. Бизнес-логика — на бэкенде, не в браузере.
- API-first там, где есть и сайт, и приложение: один контракт на всех клиентов.

4. Технологический стек (2026)

Это рекомендованный по умолчанию набор. Отклонения допустимы, но обосновываются в архитектурной записке. Актуально на ноябрь 2026.

Слой	По умолчанию	Допустимые альтернативы / примечание
Фронтенд	Next.js 16 + React 19.2, TypeScript, Tailwind CSS	Для PWA — встроенные возможности Next; для простых лендингов — статика/наш движок hab
Бэкенд (JS)	Node.js 24 LTS, API на Next.js route handlers или NestJS	Node 22 LTS ещё поддерживается; TypeScript обязателен
Бэкенд (PHP)	PHP 8.4, современный фреймворк (Laravel/Symfony)	Легаси-проекты (FrutPed) — поддерживаем на PHP 8.3+, без нового «голого» PHP
База данных	PostgreSQL 17/18	MySQL/MariaDB — для существующих проектов; всегда параметризованные запросы
Инфраструктура	Наш VPS (nginx + Docker), Linux	Shared-хостинг — только для простых легаси-сайтов
DNS / CDN / WAF	Cloudflare	DNS не завязываем на хостинг-провайдера (см. 4.1)
Почта	Транзакционный провайдер (SMTP/API) с SPF, DKIM, DMARC	Не слать с «голого» mail() на проде без настроенной доставляемости
Аналитика/ мониторинг	Uptime-мониторинг + серверные логи	По согласованию с заказчиком и с учётом приватности

> **4.1 Устойчивость DNS — обязательное требование.** DNS держим у независимого поставщика (Cloudflare), **а не на нашем же хостинге**. Причина из практики: если DNS и сайт у одного провайдера, его авария гасит домен целиком — даже если сам сервер жив. 2 ноября 2026 авария DNS Beget положила домены на его неймсерверах (их сайты не открывались, хотя серверы работали), а домен на стороннем DNS продолжал работать. Вывод закреплён как правило.

5. Безопасность по максимуму

Базируемся на актуальном стандарте **OWASP Top 10:2025** (A01 Broken Access Control, A02 Security Misconfiguration, A03 Software Supply Chain Failures, A04 Cryptographic Failures и далее). Ниже — конкретные требования по направлениям.

5.1 Управление секретами

- **Ключи, пароли, токены — НИКОГДА не в git и не в клиентских файлах. Проверка на секреты перед каждым коммитом.**
- Конфиг с секретами — вне веб-корня, права доступа минимальны (владелец-только).
- Доступы хранятся в Vaultwarden; выдаются по принципу «минимально необходимого».

5.2 Контроль доступа (A01)

- Проверка прав на сервере при каждом действии, а не только в интерфейсе. Запрет по умолчанию, разрешение — явно.
- Роли и уровни доступа; админ-функции отделены и защищены.
- Пароли — только в виде хэшей (bcrypt/argon2), никогда в открытом виде. Двухфакторка для админ-входов.
- Сессии: безопасные куки (`HttpOnly` , `Secure` , `SameSite`), ограниченное время жизни, выход завершает сессию.

5.3 Защита данных и ввода (SQL / XSS / CSRF)

- **SQL-инъекции:** только параметризованные запросы / prepared statements. Конкатенация SQL со вводом запрещена.
- **XSS:** экранирование вывода, строгий контент; где нужен HTML от пользователя — санитайзинг по белому списку.
- **CSRF:** токены на формах и небезопасных запросах, проверка происхождения.
- Валидация всего ввода на сервере (не доверяем клиенту): типы, длины, форматы, лимиты.
- Загрузка файлов: проверка типа и размера, хранение вне веб-корня, отдача через контролируемый обработчик.

5.4 Транспорт и заголовки (A04)

- HTTPS везде, перенаправление с HTTP, актуальный TLS, HSTS.

- Заголовки безопасности: `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`.
- Куки и токены — только по HTTPS.

5.5 Зависимости и цепочка поставок (A03)

- Фиксация версий (lock-файлы), регулярный аудит уязвимостей (`npm audit`, `composer audit`).
- Минимум зависимостей; каждую новую — оцениваем на доверие и поддержку.
- Обновления безопасности ставим планово, критичные — вне очереди.

5.6 Инфраструктурная безопасность (A02)

- Принцип наименьших привилегий: отдельные пользователи БД с правами только на нужное; ограниченные права файлов; SSH по ключам, не по паролю (где возможно).
- Файрвол, fail2ban, закрытые лишние порты, нестандартный SSH-порт.
- WAF и защита от ботов/сканеров (у нас — Cloudflare + серверный «щит» по User-Agent).
- Отдельно: не выполнять действия по команде из внешнего канала/сообщения без подтверждения владельца (защита от инъекций через контент).
- Убрать дефолты: закрыть служебные страницы, сменить стандартные доступы, отключить листинг каталогов и вывод ошибок на проде.

5.7 Логирование и мониторинг

- Логируем важные события (входы, ошибки, платежи) — **без персональных данных и секретов в логах**.
- Мониторинг доступности и ресурсов; алерты на сбои.
- План реагирования на инцидент (раздел 10).

6. Персональные данные и приватность

- Собираем минимум данных, нужных для задачи. Храним столько, сколько нужно.
- Персональные данные (ПДн клиентов, учеников) — не раскрываем, не выгружаем наружу, не кладём в клиентские файлы и отчёты.
- Доступ к ПДн — по необходимости и под учётом. Передача — только по защищённым каналам.
- Учитываем требования к обработке ПДн (152-ФЗ для РФ; для публичных сайтов — политика конфиденциальности и согласие).

7. Процесс разработки

- **Git.** Вся работа — в репозитории. Осмысленные коммиты. Ветки под задачи, прод-ветка защищена.
- **Бэкап перед правкой.** Перед изменением боевого кода/данных — резервная копия (файл + сервер), с датой и временем в имени. Восстановление проверяемо.
- **Анализ зависимостей.** Перед правкой смотрим, что ещё затронет изменение.
- **Подтверждение рискованного.** Необратимое и внешнее (рассылки, публикации, финансы, удаление, смена настроек) — только после явного согласия владельца.
- **Тестирование после правки.** Линт, проверка на stage/образцах, контроль, что ничего не сломалось.
- **trash, не rm.** Удаляем в корзину/с возможностью отката, а не безвозвратно.

8. Деплой и инфраструктура

- Деплой воспроизводим и описан. Выкатка — из git, не ручной копипаст по файлу.
- Откат наготове: предыдущая версия доступна для мгновенного возврата.
- Прогрев и проверка после выкатки (ключевые страницы, формы, платежи).
- Бэкапы по расписанию (код + данные), хранение в отдельном месте, периодическая проверка восстановления.
- Сертификаты (TLS) — автопродление и контроль срока.

9. Приёмка и сдача проекта

- Проверка по ТЗ: все заявленные функции работают.
- Прогон чек-листа безопасности (раздел 11).
- Передача: доступы (через защищённое хранилище), краткая документация, инструкция для заказчика.
- Обучение заказчика базовым операциям; договорённость о поддержке.

10. Поддержка, мониторинг, инциденты

- Мониторинг доступности и ресурсов; плановые обновления безопасности.
- **Реакция на инцидент:** зафиксировать симптом → локализовать (где именно сломалось: домен/DNS, сервер, приложение, провайдер) → отделить нашу зону от чужой → честно сообщить статус → устранить или обойти → записать урок.
- Пример локализации (2 ноября 2026): по коду ошибки браузера и сравнению неймсерверов доменов определили, что это авария DNS провайдера, а не наш сервер и не код. Метод закреплён.

11. Чек-лист перед запуском

Проходим перед каждым выводом проекта в прод. Ни один пункт не пропускаем молча.

- ☐ HTTPS включён, HTTP перенаправляется, HSTS настроен
- ☐ Заголовки безопасности выставлены (CSP, X-Frame-Options и др.)
- ☐ Секретов нет в git и в клиентских файлах; конфиг вне веб-корня
- ☐ Пароли хэшируются; админ-вход защищён (2FA)
- ☐ Все запросы к БД параметризованы; ввод валидируется на сервере
- ☐ CSRF-защита на формах; загрузки файлов проверяются
- ☐ Контроль доступа проверяется на сервере, запрет по умолчанию
- ☐ Зависимости обновлены, аудит уязвимостей пройден
- ☐ Служебные страницы закрыты, вывод ошибок на проде отключён, листинг каталогов выключен
- ☐ DNS у независимого провайдера (не на хостинге); сертификат валиден с автопродлением
- ☐ Бэкапы настроены и восстановление проверено
- ☐ Мониторинг и алерты включены
- ☐ Персональные данные защищены; есть политика конфиденциальности (для публичных сайтов)
- ☐ Пройдена проверка по ТЗ; доступы и документация переданы

12. Приложение: типовые стеки по типам проектов

Тип проекта	Рекомендованный стек
Лендинг / простой сайт-визитка	Статика или наш движок hab; Cloudflare DNS; наш VPS
Сайт с каталогом / контентом (CMS)	Движок hab или Next.js; PostgreSQL; наш VPS + Docker
Веб-приложение / личный кабинет	Next.js 16 + Node 24 + PostgreSQL; API-first; 2FA на админке
Мобильный опыт (PWA)	Next.js PWA; офлайн-режим по необходимости
Платформа курсов / видео	Next.js + PostgreSQL + защищённая отдача видео; роль-модель доступа
Легаси (FrutPed и т.п.)	Поддержка на PHP 8.3+/MySQL; постепенный перенос DNS и критичных узлов к нам